

Egyptian E-Learning University
Faculty of Computers & Information Technology
Department of Information Technology

NileGuide

Tourism Planning & Recommendation Platform
Final Graduation Project Documentation

Team Members

Name	ID
Yossef Abdelmoneim Abdel Hamid	2200395
Abdelrahman Tamer Mahmoud	2200406
Mohamed Ahmed Attia	2200401
Youssef Mohamed Ali	2200718
Ziad Hamdy Mohamed	2202656
Zyad Samy Mohamed	2200399
Ahmed Hassan Younes	2202660
Ahmed Hesham Abdelshakour	2200783

Supervised by

Dr. Kamal Hamza

Assistant

Eng. Mohamed Kamal

Cairo - 2026

Abstract

NileGuide is a web-based tourism planning platform for tourists visiting Egypt. The delivered system brings activity discovery, detailed activity pages, personal wishlists, schedule planning, map viewing, user profiles, admin management, reporting, and an AI knowledge assistant into one application. The frontend was implemented using Angular Standalone components with Tailwind CSS and Flowbite. The backend was implemented as an ASP.NET Core 8 Web API connected to SQL Server through Entity Framework Core, with JWT-based role access for tourists and administrators. A separate FastAPI service provides the RAG module used by the chatbot and by the admin file-management panel. In the final version, the AI scope moved from a general recommendation idea to a practical knowledge-base assistant: administrators upload PDF, DOCX, CSV, or TXT files; the service extracts and chunks their content; embeddings are indexed in Qdrant; and answers are generated from retrieved context. This report describes the final implemented system rather than only the first-term plan. It explains the real user and admin flows, the implemented endpoints, database design, AI/RAG behavior, screenshots, testing approach, and the limitations that remain for future work.

Acknowledgments

The NileGuide team would like to thank the project supervisors for their guidance during the analysis, implementation, and review phases. We also thank the Faculty of Computers & Information Technology at the Egyptian E-Learning University for providing the academic framework for this graduation project. The final version of the project was shaped by continuous feedback, testing, and adjustments made during frontend, backend, database, deployment, and AI integration work.

Contents

Abstract

Acknowledgments

List of Figures

List of Tables

List of Abbreviations

Chapter 1 - Introduction

Chapter 2 - Literature Review / Related Work

Chapter 3 - Proposed System

Chapter 4 - Implementation

Chapter 5 - Testing & Evaluation

Chapter 6 - Results & Discussion

Chapter 7 - Conclusion & Future Work

References

Appendices

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
AI	Artificial Intelligence
RAG	Retrieval-Augmented Generation
JWT	JSON Web Token
DTO	Data Transfer Object
EF Core	Entity Framework Core
SQL	Structured Query Language
UI	User Interface
UX	User Experience
SPA	Single Page Application
SSR	Server-Side Rendering
LLM	Large Language Model

Table 1. Abbreviations used in the NileGuide report

Chapter 1

Introduction

1.1 Project Background

Egypt has a wide range of historical, cultural, and entertainment activities, but tourists still need to move between many sources before they can decide what to do, where to go, and how to organize their time. In the first-term documentation, the problem was described as scattered information and weak planning tools. The final implementation kept that core idea and turned it into a working web system where tourists can discover activities, open a full details page, save activities, and build a schedule from one place.

NileGuide is therefore not just a static activities website. It combines content browsing, filtering, scheduling, map support, admin operations, reporting, and an AI assistant. The admin side is important because tourism content changes; activities, users, reports, and AI knowledge files need to be managed after deployment without changing frontend code.

1.2 Problem Statement

- Tourists often search across social media, blogs, travel agencies, and booking websites before they can compare Egyptian activities.
- Basic listing websites do not always connect discovery with personal planning tools such as wishlist and schedule.
- Administrators need a controlled dashboard to manage activity data, users, images, reports, and AI knowledge files.
- A chatbot should not answer from general memory only; it should use project-controlled documents and return answers grounded in the uploaded knowledge base.

1.3 Project Objectives

- Provide a tourist-facing web interface for browsing Egyptian activities by city, category, search keyword, and price sorting.
- Provide dynamic activity details pages that load activity information, image gallery, providers, opening hours, and reviews from the backend API.
- Allow tourists to maintain a wishlist and a date/time schedule that can be exported as a PDF.
- Provide a profile page where the tourist can update personal preferences and profile picture.
- Provide an administrator dashboard for users, activities, reports, and AI file management.
- Integrate a RAG-based assistant that uses uploaded project documents instead of relying only on a generic AI response.
- Deploy the core web/API layers using a maintainable frontend-backend architecture.

1.4 Project Scope in the Final Version

The final report documents the features that are actually delivered in the current project version. Some first-term ideas were adjusted during implementation. For example, the AI layer changed from a planned recommendation concept into a working RAG assistant and file-management module. Payment, real booking confirmation, full personalized trip optimization, refresh-token UX, and Remember Me are not treated as delivered features in this report.

Area	Final status	Notes
Activity discovery and filtering	Implemented	Activities list, filters, search, sorting, pagination and dynamic details are connected to backend APIs.
Wishlist	Implemented	Tourists can save/remove activities and retrieve wishlisted IDs.
Schedule	Implemented	Tourists can add activities with date/time, view totals, remove items and export PDF. It prevents duplicate scheduled activity records, but full time-conflict optimization is future work.
Admin dashboard	Implemented	Users, activities, reports and AI file management are covered.
AI/RAG	Implemented with changed scope	The delivered AI module is a document-based RAG assistant with upload/list/update/delete and chat endpoints.
Booking/payment	Out of scope	The system shows provider links such as Viator/GetYourGuide/Tripadvisor, but it does not process payments or confirm bookings.
Refresh token / Remember Me	Out of current documented scope	The final user-facing documentation covers JWT access token and role-based access only.

Table 2. Final implementation scope compared with the first-term plan

Chapter 2

Literature Review / Related Work

2.1 Overview

Related systems in tourism planning usually focus on either itinerary generation, attraction recommendation, travel package recommendation, or chatbot support. NileGuide was built with a more operational direction: it combines discovery, personal planning, admin content management, and AI knowledge assistance in one deployable web project.

Study	Main idea	Useful point	Gap
Chang et al. (2016)	Automatic travel itinerary planning using multi-factor scoring	Reduced planning effort through itinerary construction	Domestic scope and limited operational features
Hassannia et al. (2019)	Smart tourism agency with hybrid recommendation	Combined package recommendation with real-time data communication	Evaluation was limited and focused mainly on packages
Srisawatsakul & Boontarig (2020)	Attraction recommendation from Instagram photos	Used content-based filtering and visual labels	Small dataset and privacy concerns
Kumar et al. (2020)	Travel recommender with chatbot and web-scraped data	Included conversational support and travel aggregation	No strong reported evaluation and risk of scraping instability

Table 3. Summary of related tourism planning and recommendation work

2.2 Research Gap Addressed by NileGuide

From the reviewed work, the main gap is not only the recommendation algorithm. A practical tourism platform also needs authentication, role-based access, content operations, reliable APIs, database design, image handling, reporting, and a way to update the assistant knowledge base. NileGuide addresses these platform-level requirements by connecting an Angular frontend, an ASP.NET Core backend, a SQL database, and a separate FastAPI RAG module.

Chapter 3

Proposed System

3.1 System Overview

The proposed system is a three-part architecture. The first part is the Angular web application used by tourists and admins. The second part is the ASP.NET Core backend API that stores and serves the core business data. The third part is the AI/RAG service, built separately using FastAPI, MongoDB and Qdrant, so that document processing and vector search do not overload the main .NET API.

NileGuide System Architecture

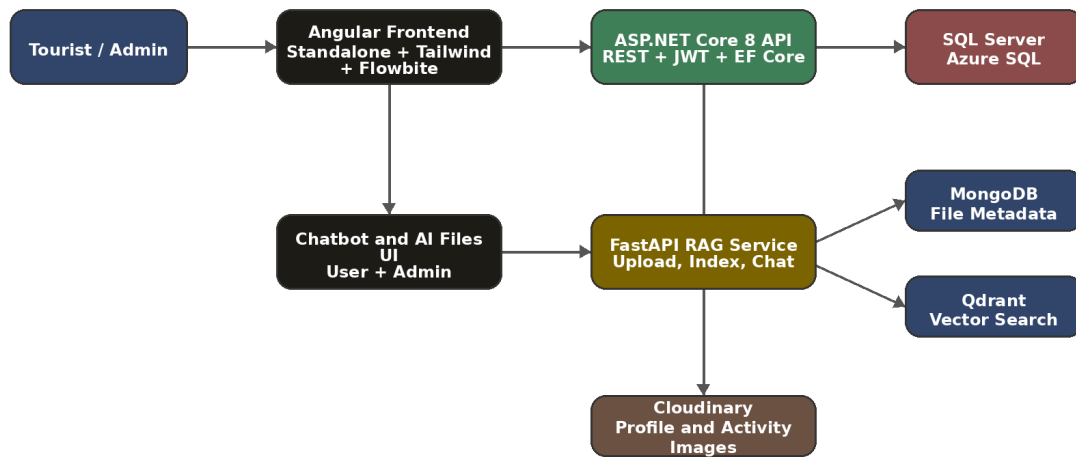


Figure 1. NileGuide high-level system architecture

3.2 User Roles

Role	Responsibilities
Tourist	Registers/logs in, browses activities, opens details, uses wishlist, schedule, map, profile, and chatbot.
Admin	Manages users, activities, reports, and AI knowledge files from the dashboard.
Visitor/Guest	Can view public pages such as home, login, register, contact, forgot password, and reset password.

Table 4. System roles and responsibilities

3.3 Functional Requirements

ID	Requirement
FR-01	Register tourist account with full name, email, password, nationality and date of birth.
FR-02	Login using email and password and store JWT access token and role in the frontend.
FR-03	Protect tourist-only pages using tourist guard and admin-only pages using admin guard.
FR-04	Browse activities with category, city, search, sorting and pagination.
FR-05	Open dynamic activity details using /activities/:id.
FR-06	Add/remove activities to/from wishlist and retrieve wishlisted IDs.
FR-07	Add/remove activities to/from schedule and display date, start time, end time, duration and cost.
FR-08	Export the tourist schedule as PDF.
FR-09	Show activity locations on a map with itinerary context.
FR-10	Allow users to update profile data, preferences and profile picture.
FR-11	Allow admins to manage users and activity status/roles where applicable.
FR-12	Allow admins to create, update and delete activities with multipart image upload.
FR-13	Show admin reports for activity views, user growth, categories and top activities.
FR-14	Allow AI file upload, list, update and delete for the RAG knowledge base.
FR-15	Allow users to ask the chatbot and receive answers generated from retrieved documents.

Table 5. Functional requirements implemented or represented in the final version

3.4 Non-Functional Requirements

- Security: JWT-based access token is sent with protected API requests and admin operations require an admin role.

- Maintainability: the frontend separates pages, services, guards and interceptors; the backend separates controllers, DTOs, services and EF Core models.
- Usability: pages use a consistent dark visual style, clear navigation, filters, modals and feedback messages.
- Scalability of AI knowledge: the AI service stores document metadata separately from vectors and can re-index files when they are updated.
- Responsiveness: the Angular UI is designed for desktop and smaller screens using Tailwind utility classes.
- Deployment readiness: the frontend and backend are separated, and the backend exposes Swagger for API inspection.

3.5 Main Workflows

Tourist Workflow in the Delivered System

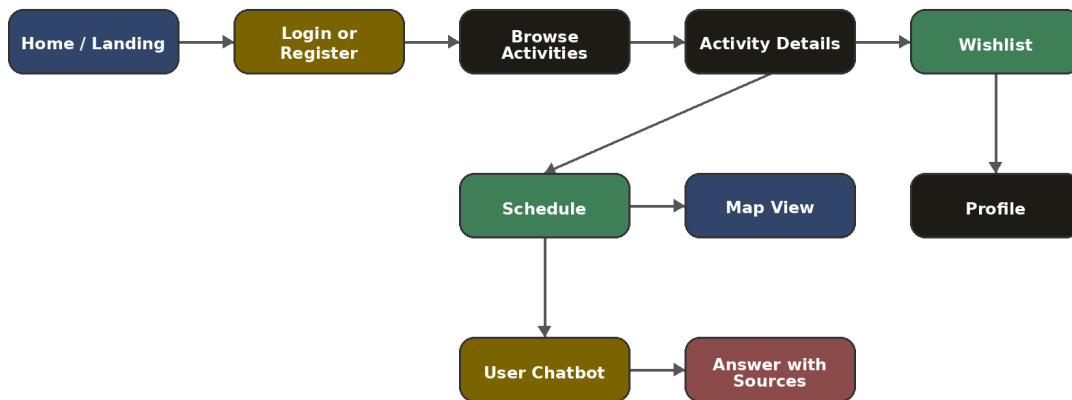


Figure 2. Tourist workflow from discovery to chatbot support

Admin Workflow

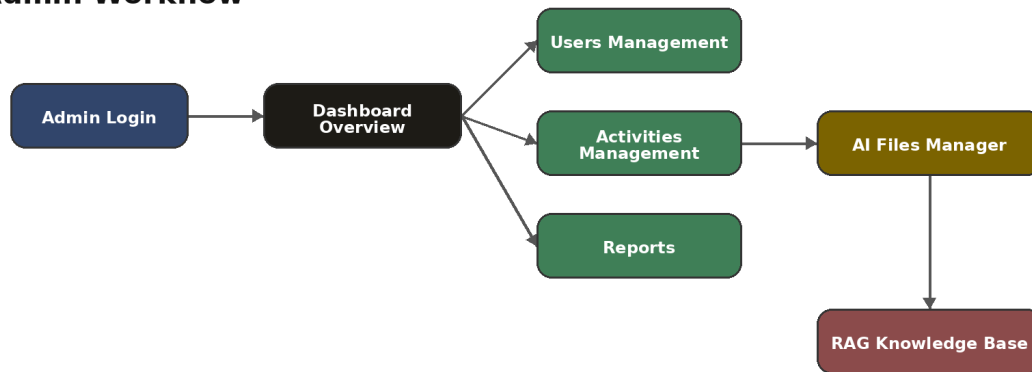
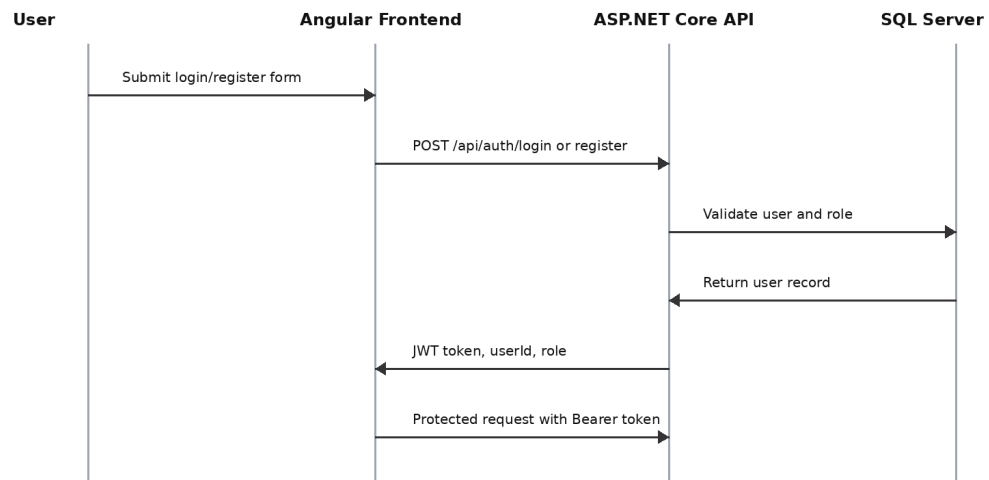


Figure 3. Admin workflow through dashboard modules

Authentication and Authorization Sequence



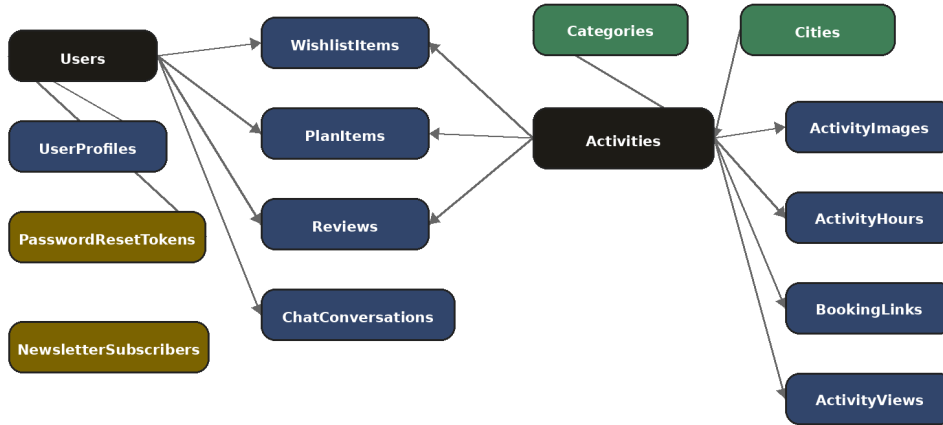
Final report scope: JWT access token and role-based route protection. Remember Me and refresh-token UX are not treated a

Figure 4. JWT authentication and protected request sequence

3.6 Database Design

The backend database uses SQL Server through Entity Framework Core. The main relational data belongs to users, profiles, activities, cities, categories, images, hours, providers, wishlist items, schedule items, reviews, activity views, newsletter subscribers and chat conversations. The RAG module uses MongoDB and Qdrant separately because its data is document/vector oriented rather than relational.

Core Database Design



The diagram shows the relationships used by discovery, profile, wishlist, schedule, review, reporting, and chat modules. Refresh tokens exist in t

Figure 5. Simplified ERD for the main SQL Server entities

Chapter 4

Implementation

4.1 Frontend Implementation

The frontend is an Angular Standalone application. It uses Angular routing, standalone components, Tailwind CSS, Flowbite, Flowbite Datepicker, Font Awesome, ngx-toastr, ngx-spinner, jsPDF and ApexCharts. The code is organized by feature folders: auth, home, activities, details, wishlist, schedule, map, profile, contact, and admin dashboard modules. Services are responsible for API calls; guards protect routes; and interceptors handle common request behavior.

Frontend area	Implementation
Technology	Angular 21 Standalone, Angular Router, SSR/hydration support
Styling	Tailwind CSS, Flowbite components, dark theme UI
State/API	Feature services, localStorage token, RxJS observables
Feedback	ngx-toastr messages and loading states
Exports	jsPDF and autoTable for schedule PDF export
Charts	ApexCharts used in admin reports

Table 6. Frontend implementation summary

4.2 Frontend Routes

Route	Purpose
/home	Landing page
/auth/login	Login page
/auth/register	Register page
/auth/forgot-password	Forgot password page
/auth/reset-password	Reset password page
/activities	Tourist activities list
/activities/:id	Dynamic activity details
/wishlist	Tourist wishlist
/schedule	Tourist schedule
/map	Map view
/profile	Tourist profile

/contact	Contact page
/dashboard	Admin dashboard shell
/dashboard/users-management	Admin users management
/dashboard/activities-management	Admin activities management
/dashboard/reports	Admin reports

Table 7. Angular routes used by the final frontend

4.3 Backend Implementation

The backend is an ASP.NET Core 8 Web API. It registers EF Core with SQL Server, JWT authentication, role authorization, CORS for the Angular origins, Swagger documentation, rate limiting, Cloudinary image handling, email services, SignalR dashboard hub, and centralized exception handling middleware. Controllers expose a REST API to the frontend, while services contain the business logic.

Backend area	Implementation
Framework	.NET 8 / ASP.NET Core Web API
Database	SQL Server through Entity Framework Core
Security	JWT Bearer authentication and AdminOnly policy
Images	Cloudinary service for profile and activity images
Email	SMTP/MailKit for reset-password and newsletter scenarios
API documentation	Swagger UI at /swagger
Realtime support	SignalR dashboard hub is mapped at /hubs/dashboard

Table 8. Backend implementation summary

4.4 API Documentation

Module	Endpoints	Purpose
Auth	POST /api/auth/register, POST /api/auth/login, POST /api/auth/forgot-password, POST /api/auth/verify-reset-code, POST /api/auth/reset-password	Registration, login and password reset flow

Activities	GET /api/Activities, GET /api/Activities/{id}, GET/POST /api/activities/{activityId}/reviews	Activity list, details and reviews
Lookups	GET /api/Categories, GET /api/Cities	Dynamic categories and cities
Wishlist	GET /api/wishlist, GET /api/wishlist/activity- ids, POST /api/wishlist/{activityId}, DELETE /api/wishlist/{activityId}, GET /api/wishlist/status/{activityId}	Personal saved activities
Plan/Schedule	GET /api/plan, POST /api/plan/items, DELETE /api/plan/items/{planItemId}, GET /api/plan/activity-ids	Scheduled activities
Profile	GET /api/users/me/profile, PUT /api/users/me/profile, POST /api/users/me/profile-picture, DELETE /api/users/me/profile-picture	User profile and image operations
Admin users	GET /api/users, GET /api/users/{id}, POST /api/users, PATCH /api/users/{id}/role, PATCH /api/users/{id}/status, DELETE /api/users/{id}	Admin user operations
Admin activities	GET /api/admin/activities, POST /api/admin/activities, PUT /api/admin/activities/{id}, DELETE /api/admin/activities/{id}	Admin activity CRUD with multipart/form- data
Reports	GET /api/reports/activity-views, GET /api/reports/user-growth, GET /api/reports/activities-by-category, GET /api/reports/top-activities	Admin analytics
Chat conversations	POST /api/chat/conversations, GET /api/chat/conversations, DELETE /api/chat/conversations/{conversationId}	Stores user conversation identifiers/history metadata

Table 9. Main ASP.NET Core API endpoints

4.5 AI/RAG Implementation

The AI component is implemented as a separate FastAPI project. It is not a simple frontend-only chatbot. The admin uploads documents; the service saves asset metadata, extracts text, creates chunks, stores chunk records in

MongoDB, indexes embeddings in Qdrant, and uses an OpenAI-compatible generation client to answer user questions from retrieved context. The frontend contains both the tourist chatbot panel and the admin AI file panel.

Function	Endpoint	Behavior
Upload files	POST /api/v1/data/multi-upload	Accepts multiple PDF, DOCX, CSV and TXT files with chunk/overlap settings.
List files	GET /api/v1/data/files	Returns file IDs, names, sizes and upload timestamps.
Update file	PUT /api/v1/data/file/{file_id}	Replaces the file, deletes old chunks/vectors and re-indexes.
Delete file	DELETE /api/v1/data/file/{file_id}	Removes file metadata, chunks and vectors.
Chat	POST /api/v1/nlp/chat	Retrieves relevant chunks and generates an answer with sources.

Table 10. FastAPI RAG endpoints

RAG File Processing and Chat Flow

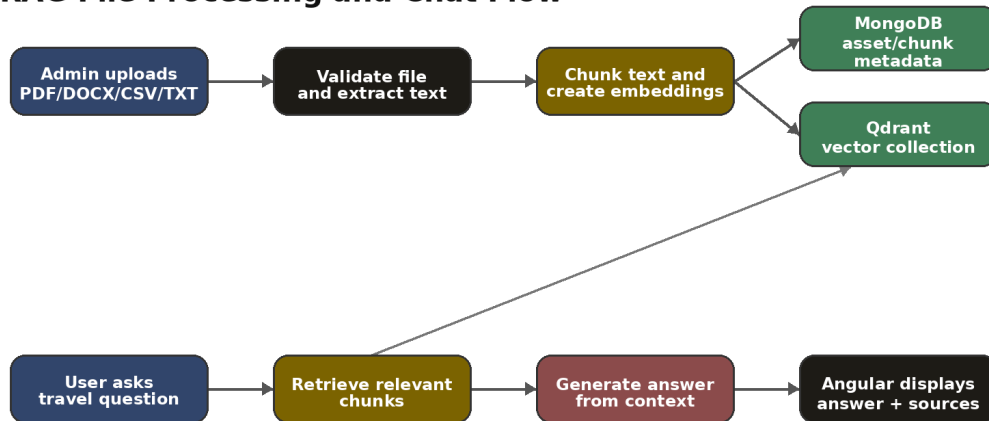


Figure 6. RAG implementation flow for upload and chat

4.6 Deployment View

The production-oriented structure separates the web frontend, the ASP.NET Core API and the AI/RAG service. The ASP.NET Core API is configured with Azure-friendly settings such as forwarded headers, CORS for

the frontend domain, Swagger, retry-on-failure SQL Server configuration, and HTTPS redirection. The frontend base API URL points to the deployed backend API. The AI service can be hosted independently because it has different runtime dependencies and storage needs.

Deployment View

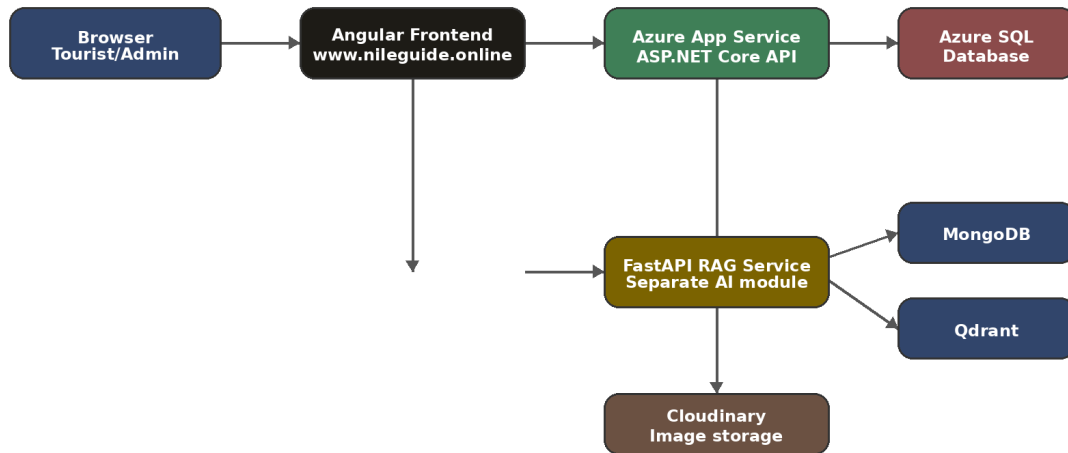


Figure 7. Deployment structure for NileGuide components

Chapter 5

Testing & Evaluation

5.1 Testing Approach

Testing was performed around complete user scenarios instead of isolated pages only. The team checked that public pages load, tourists can authenticate and reach protected pages, admin routes are blocked for non-admin users, activity cards lead to dynamic details, and admin actions call the expected API endpoints. API testing was also supported through Swagger and Postman-style endpoint inspection.

5.2 Manual Test Cases

ID	Scenario	Expected behavior	Result
TC-01	Open home page	Home loads with hero, featured content and navigation	Passed in screenshot run
TC-02	Login as tourist	JWT is stored and tourist pages are accessible	Passed
TC-03	Open activities	Cards, filters and pagination display data from API	Passed
TC-04	Open activity details	Details page loads by activity id with images/providers/hours	Passed
TC-05	Wishlist add/remove	Wishlist IDs and saved cards are retrieved	Passed
TC-06	Schedule page	Schedule shows date/time, duration, cost and export option	Passed
TC-07	Profile update	Profile form and profile picture state load for the user	Passed
TC-08	Admin users	Admin can view/search user records and actions	Passed
TC-09	Admin activities	Admin can view and manage activity records	Passed
TC-10	Reports	Charts and report summaries are displayed from backend data	Passed with available data
TC-11	AI file upload panel	Admin file upload/list UI opens correctly	Passed
TC-12	Chatbot	User chatbot opens and sends questions to AI flow	Passed

Table 11. Representative manual test cases

5.3 Screenshot Verification

A Playwright-based documentation screenshot script was used to capture the final interface states. Public pages were captured without authentication, tourist pages were captured using a tourist account, and admin pages were captured using an admin account. This process was useful because it verified route access, page rendering and visible UI states at the same time.

5.4 Known Limitations Found During Review

- Schedule supports date/time and export, but a full time-conflict optimizer is not part of the final delivered scope.
- The system displays external booking provider links, but it does not handle payment, seat availability, or booking confirmation.
- The RAG assistant depends on the quality and coverage of uploaded knowledge files; if no relevant chunks are found, it returns a fallback message.
- Reports depend on data captured by the backend. Empty or low-volume datasets will produce limited analytics.
- Refresh token and Remember Me are not presented as implemented user-facing features in the final report.

Chapter 6

Results & Discussion

6.1 Delivered Result

The final delivered result is a working tourism planning website with both tourist and admin experiences. The following walkthrough explains each captured screen exactly as it appears in the final interface: what the user sees, what action can be taken from that page, and how the screen connects to the implemented frontend, backend or AI flow.

6.2 Home page

The home page is the public starting point of NileGuide. The visitor sees the main promise of the platform: planning an Egyptian trip from one place. The hero section uses a pyramids background, a Get Started call-to-action, and quick indicators such as 200+ experiences, 4.9 average rating and 50,000+ travelers. Below the hero, the page shows city cards for Cairo, Luxor, Aswan and Giza with activity counts. This screen is important because it introduces the tourism catalogue before the user logs in.

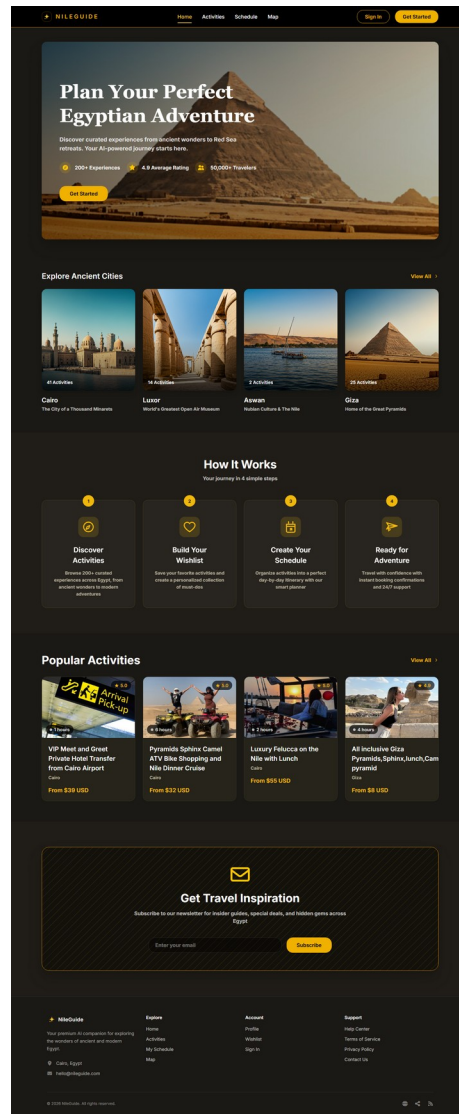


Figure 8. Home page

6.3 Login page

The login screen is where an existing user enters email and password. The page also gives access to Forgot Password, Sign Up and Back to Home. After a successful login, the frontend receives a JWT access token and role from the backend, stores the login state locally, then routes the user based on the account role. A tourist continues to the user website, while an admin can reach the dashboard.

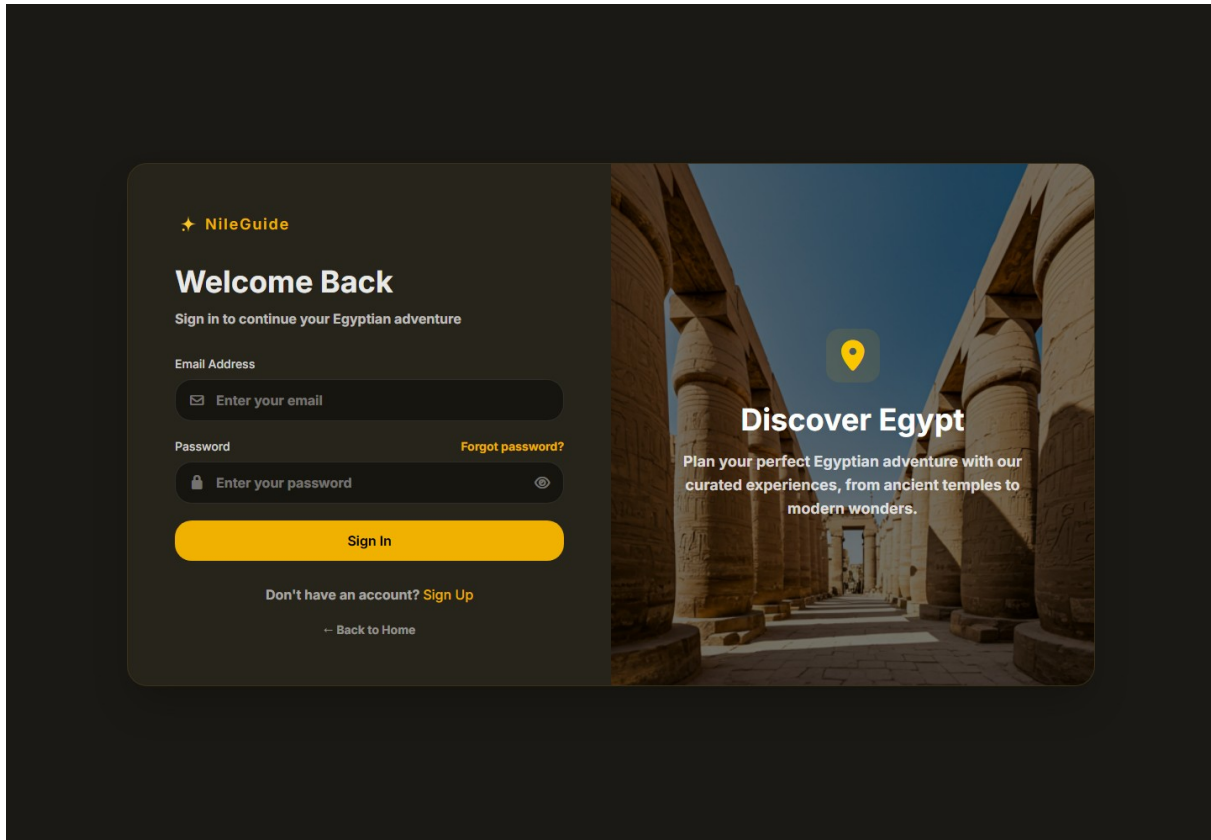
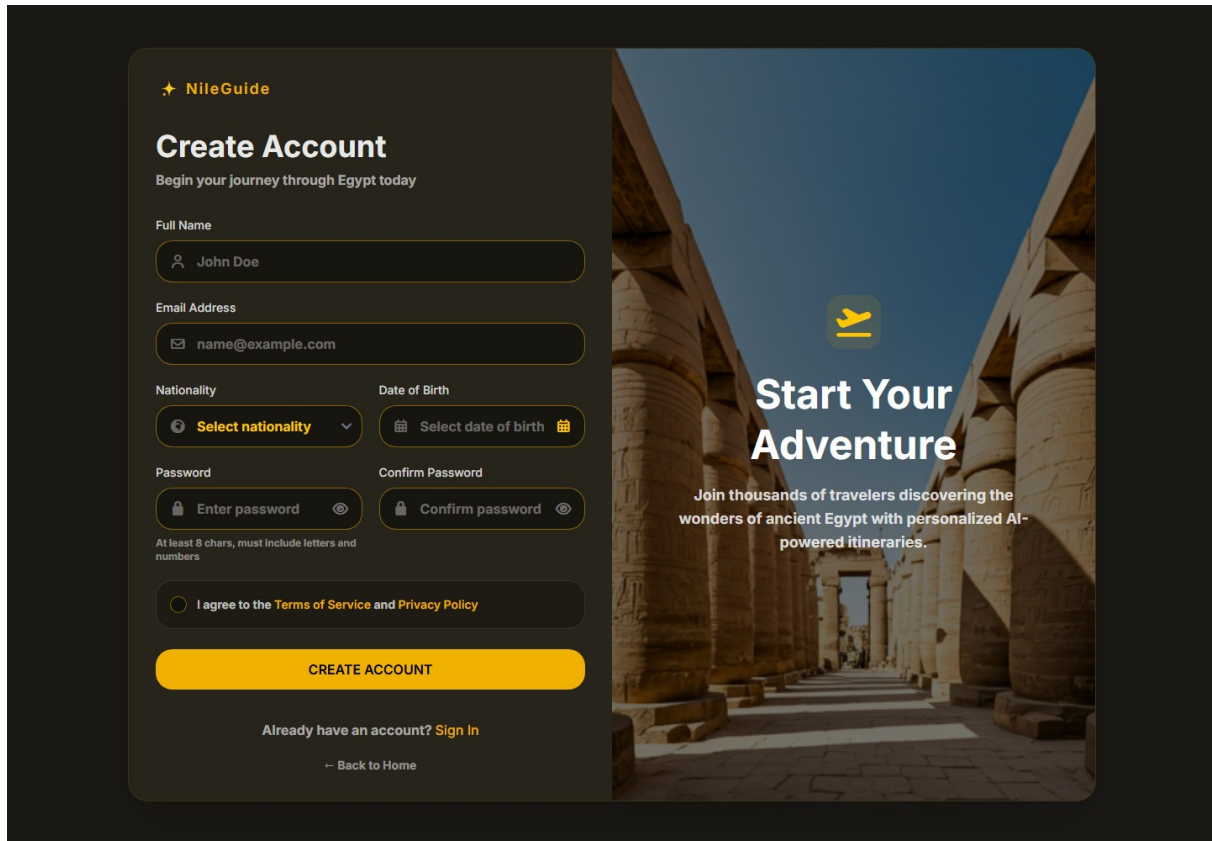


Figure 9. Login page

6.4 Register page

The register screen creates a new tourist account. The form contains full name, email, nationality, date of birth, password, confirm password and agreement to the terms and privacy policy. This matches the final backend register flow because date of birth and nationality are collected during registration instead of being added later only from the profile page.



The image shows a registration form for NileGuide. The form is on a dark background with yellow accents. It includes fields for Full Name, Email Address, Nationality, Date of Birth, Password, and Confirm Password. There is a checkbox for agreeing to terms and a 'CREATE ACCOUNT' button. A 'Sign In' link is also present. The background of the form area shows a photograph of ancient Egyptian columns.

NileGuide

Create Account

Begin your journey through Egypt today

Full Name

Email Address

Nationality

Date of Birth

Password

Confirm Password

At least 8 chars, must include letters and numbers

☐ I agree to the [Terms of Service](#) and [Privacy Policy](#)

CREATE ACCOUNT

Already have an account? [Sign In](#)

[Back to Home](#)

Start Your Adventure

Join thousands of travelers discovering the wonders of ancient Egypt with personalized AI-powered itineraries.

Figure 10. Register page

6.5 Forgot password page

This is the first step of the password recovery flow. The user enters the email address of the account and clicks Send Reset Code. The screen makes the process clear with a two-step progress header: Request Code first, then Reset Password. It also includes a Back to Login option so the user can return without completing the reset process.

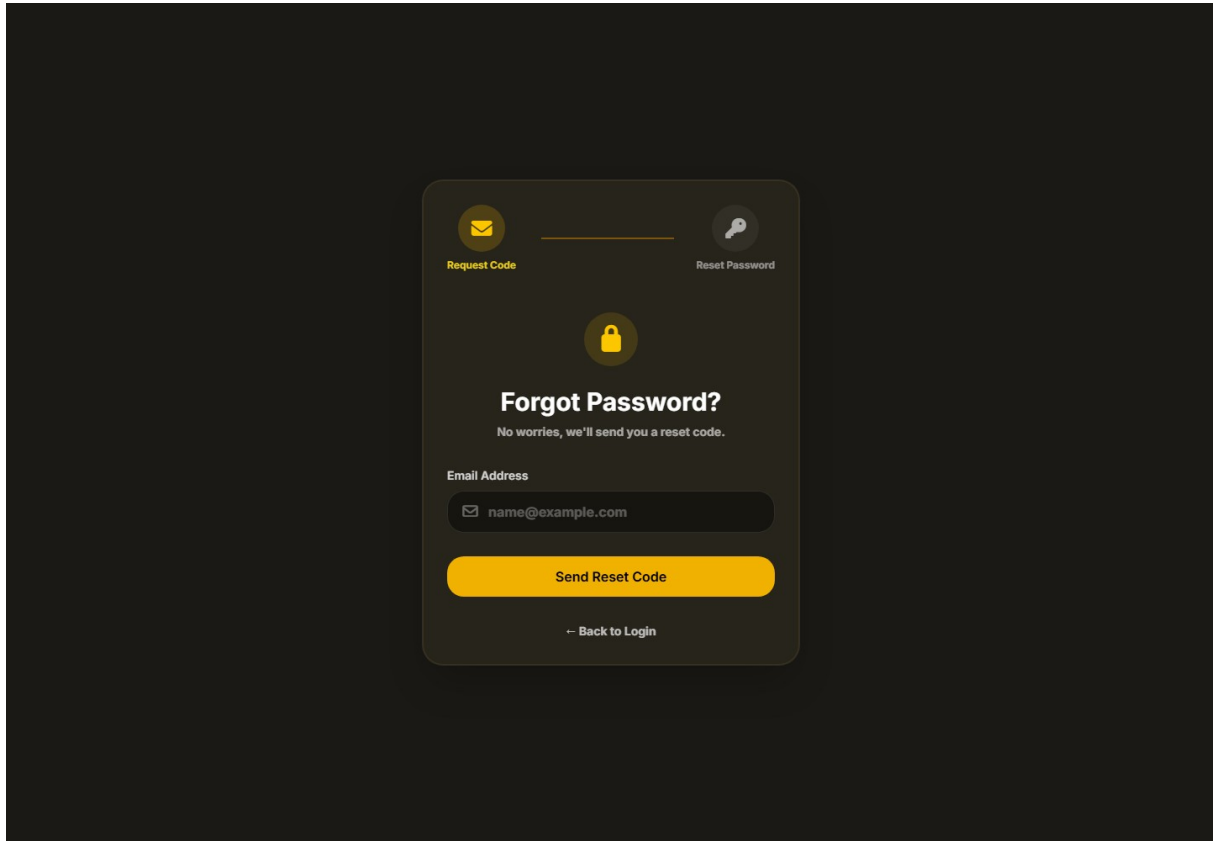


Figure 11. Forgot password page

6.6 Reset password page

This is the second step of password recovery. After the reset code is sent, the user enters the email address, the six-digit reset code, a new password and confirmation. The user can resend the code if it was not received. The backend validates the code and updates the password; the frontend keeps the screen focused on one task instead of mixing it with login.

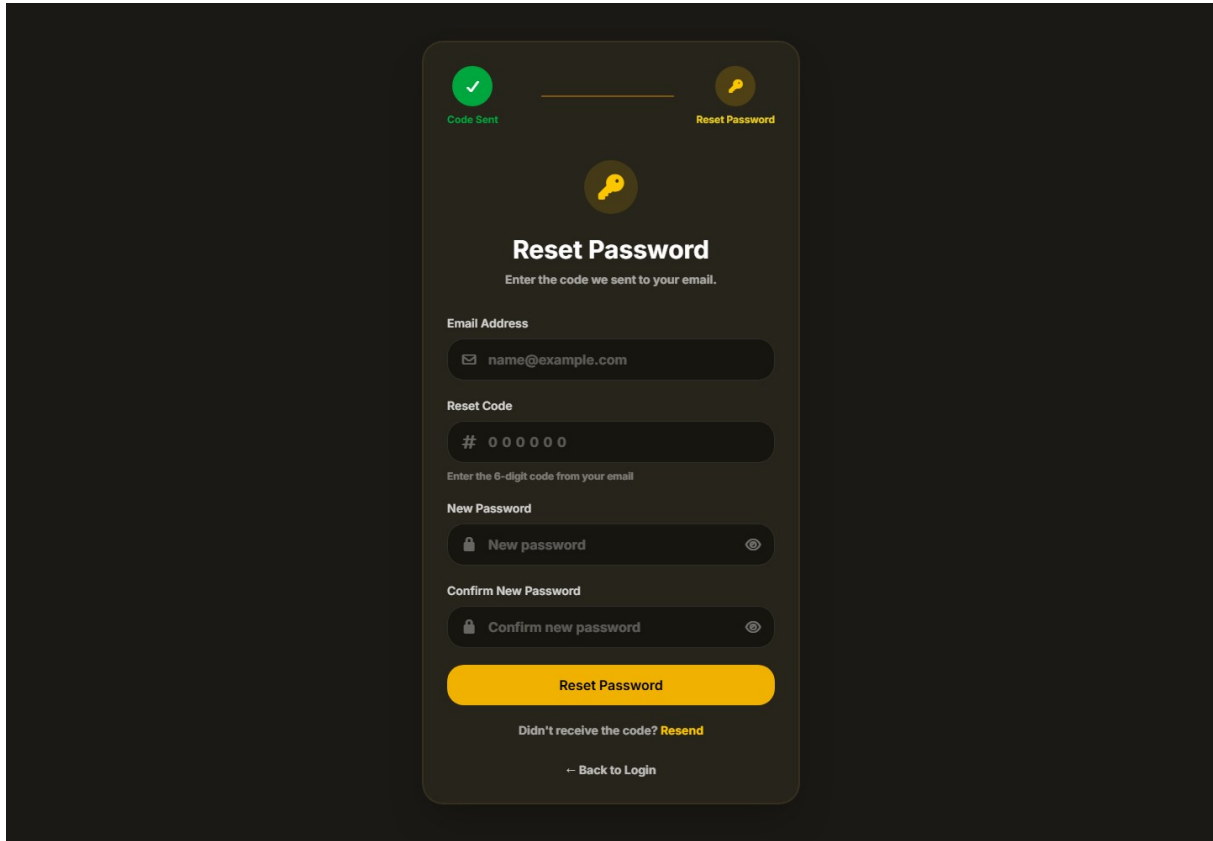


Figure 12. Reset password page

6.7 Activities page

This page is the main discovery screen for tourists. The screenshot shows a searchable list of 104 activities, category filters on the left, city filters below them, and a sort dropdown. Each activity card contains an image, category, rating, activity title, required document note, short description, opening and closing time, price, booking-source icons, Add To Plan and View Details. The user can discover activities, save them to wishlist using the heart icon, add them to the schedule, or open the dynamic details page.

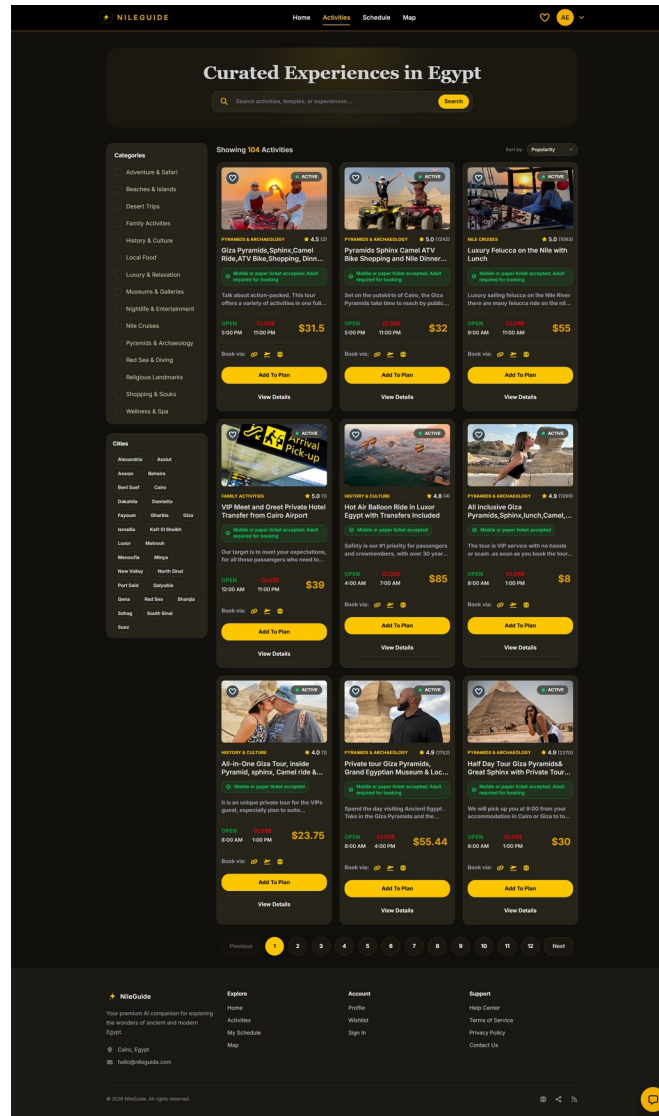
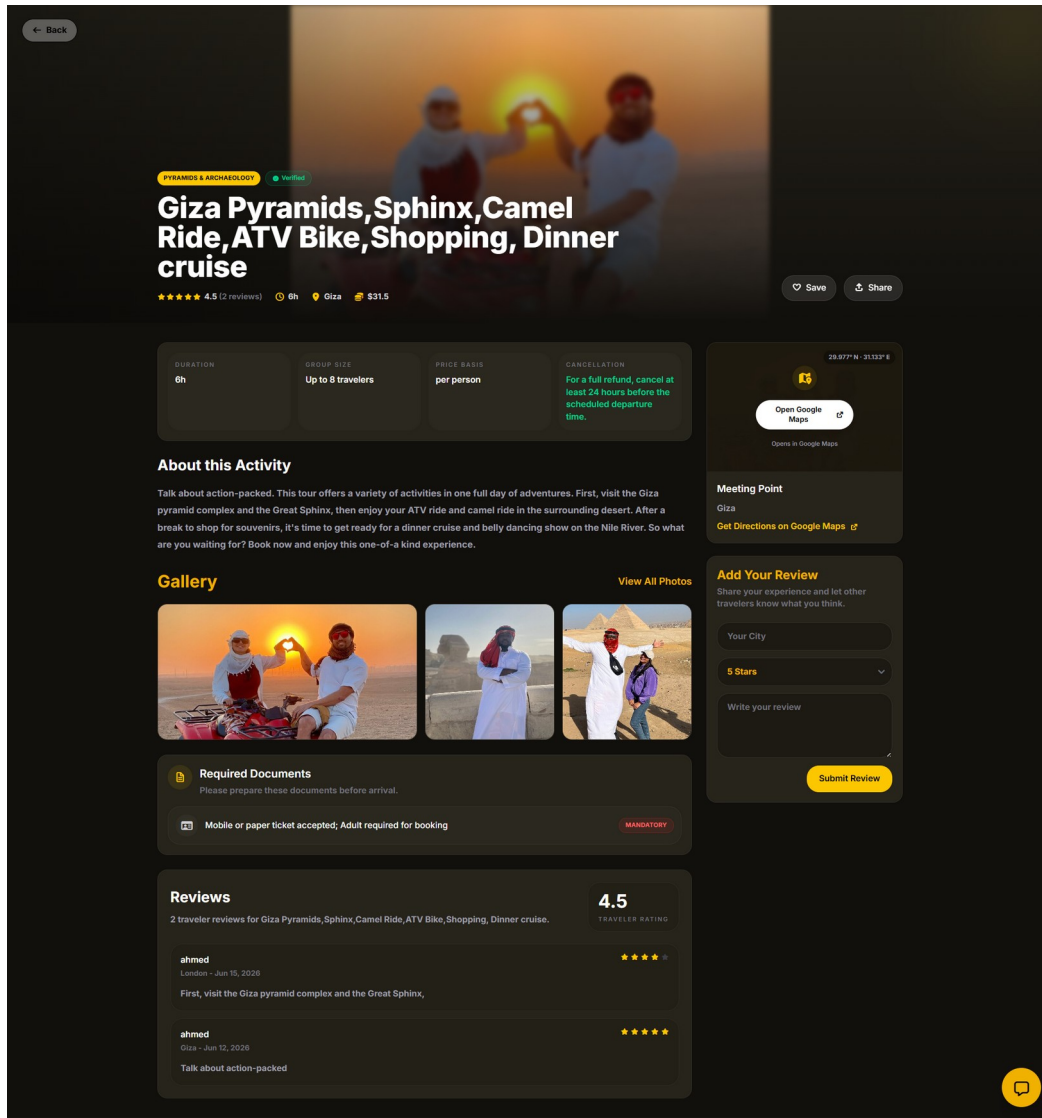


Figure 13. Activities page

6.8 Activity details page

The details page opens for one selected activity using the route `/activities/:id`. The screenshot shows the selected Giza activity with a large hero image, category label, verified badge, title, rating, duration, city and price. The page also provides Save and Share actions, summary cards for duration, group size, price basis and cancellation, an About section, a photo gallery, required documents, traveler reviews, Google Maps meeting point and an Add Your Review form. This proves that the activity page is built from the backend activity id and is not a static page.



← Back

Giza Pyramids, Sphinx, Camel Ride, ATV Bike, Shopping, Dinner cruise

★★★★★ 4.5 (2 reviews) 6h Giza \$31.5 Save Share

Duration: 6h **Group Size:** Up to 8 travelers **Price Basis:** per person **Cancellation:** For a full refund, cancel at least 24 hours before the scheduled departure time.

About this Activity

Talk about action-packed. This tour offers a variety of activities in one full day of adventures. First, visit the Giza pyramid complex and the Great Sphinx, then enjoy your ATV ride and camel ride in the surrounding desert. After a break to shop for souvenirs, it's time to get ready for a dinner cruise and belly dancing show on the Nile River. So what are you waiting for? Book now and enjoy this one-of-a-kind experience.

Gallery [View All Photos](#)

Required Documents
Please prepare these documents before arrival.

Mobile or paper ticket accepted; Adult required for booking **MANDATORY**

Reviews **4.5** TRAVELER RATING

2 traveler reviews for Giza Pyramids, Sphinx, Camel Ride, ATV Bike, Shopping, Dinner cruise.

ahmed
London - Jun 15, 2020
★★★★★
First, visit the Giza pyramid complex and the Great Sphinx,

ahmed
Giza - Jun 12, 2020
★★★★★
Talk about action-packed

Add Your Review
Share your experience and let other travelers know what you think.

Your City
5 Stars
Write your review
Submit Review

Meeting Point
Giza
[Get Directions on Google Maps](#)

29.877° N 31.133° E
[Open Google Maps](#)
Opens in Google Maps

Figure 14. Activity details page

6.9 Wishlist page

The wishlist page shows the activities that the tourist has saved for later. In the screenshot there are two saved activities, a Clear All action, and two full activity cards. Each saved card still keeps the important actions: the user can remove it using the active heart, add it to the plan, or open View Details before deciding. This feature separates interest from scheduling; saving something does not automatically add it to the trip plan.

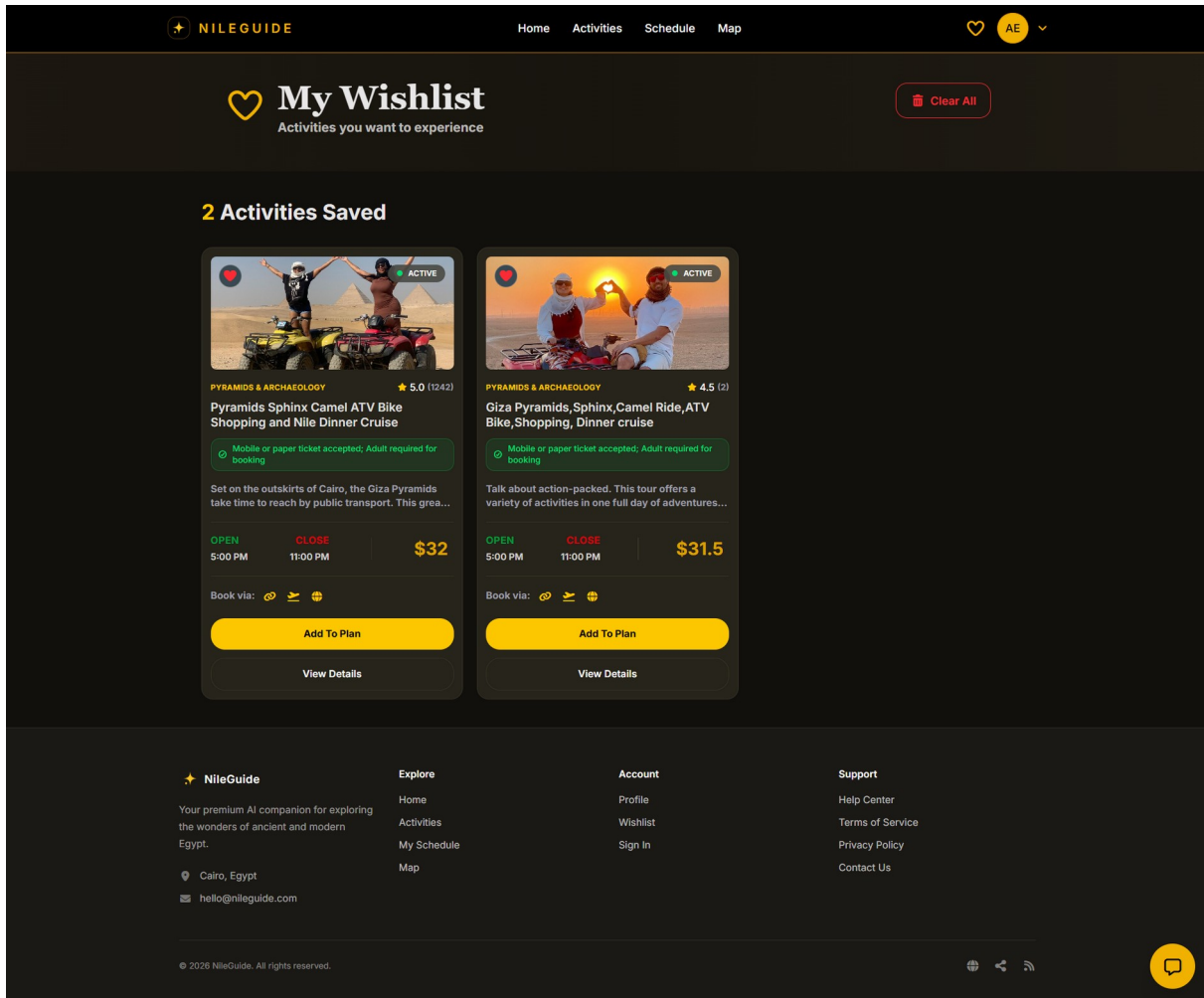


Figure 15. Wishlist page

6.10 Schedule page

The schedule page is the tourist trip plan. It summarizes the selected activities with Total Activities, Total Duration and Total Cost, then displays a table with date, time range, activity, location, opening time, closing time, duration, cost and row actions. The user can export the schedule, add another activity, open a related activity/provider action, or delete an item. This screen shows how discovery turns into an organized itinerary.


NILEGUIDE

[Home](#)
[Activities](#)
[Schedule](#)
[Map](#)


AE


My Egypt Schedule
Export

Total Activities
2

Total Duration
13h

Total Cost
\$88

TIME	ACTIVITY	LOCATION	OPEN	CLOSE	DURATION	COST	ACTIONS
Jun 25, 2026 9:00 AM to 1:00 PM	All inclusive Giza Pyramids, Sphinx, lunch, Camel, inside pyramid The tour is VIP service with no hassle or scam .as soon as you book the tour, our...	Giza	8:00 AM	1:00 PM	4h	\$8	  
Jun 25, 2026 1:30 PM to 10:30 PM	Private Day Tour To El Fayoum From Cairo Private Day Tour To El Fayoum From Cairo Escape the hustle and bustle of Cairo for...	Cairo	8:00 AM	5:00 PM	9h	\$80	  

+ Add New Activity


NileGuide
 Your premium AI companion for exploring the wonders of ancient and modern Egypt.
 Cairo, Egypt
 hello@nileguide.com

Explore
[Home](#)
[Activities](#)
[My Schedule](#)
[Map](#)

Account
[Profile](#)
[Wishlist](#)
[Sign In](#)

Support
[Help Center](#)
[Terms of Service](#)
[Privacy Policy](#)
[Contact Us](#)

© 2026 NileGuide. All rights reserved.






Figure 16. Schedule page

6.11 Map page

The map page visualizes the schedule geographically. The screenshot shows a Google Map with markers and an itinerary panel on the right containing the two planned activities with city and date. There is also an All Cities filter. The purpose of this screen is to help the tourist understand where the planned activities are located instead of reading the schedule as a table only.

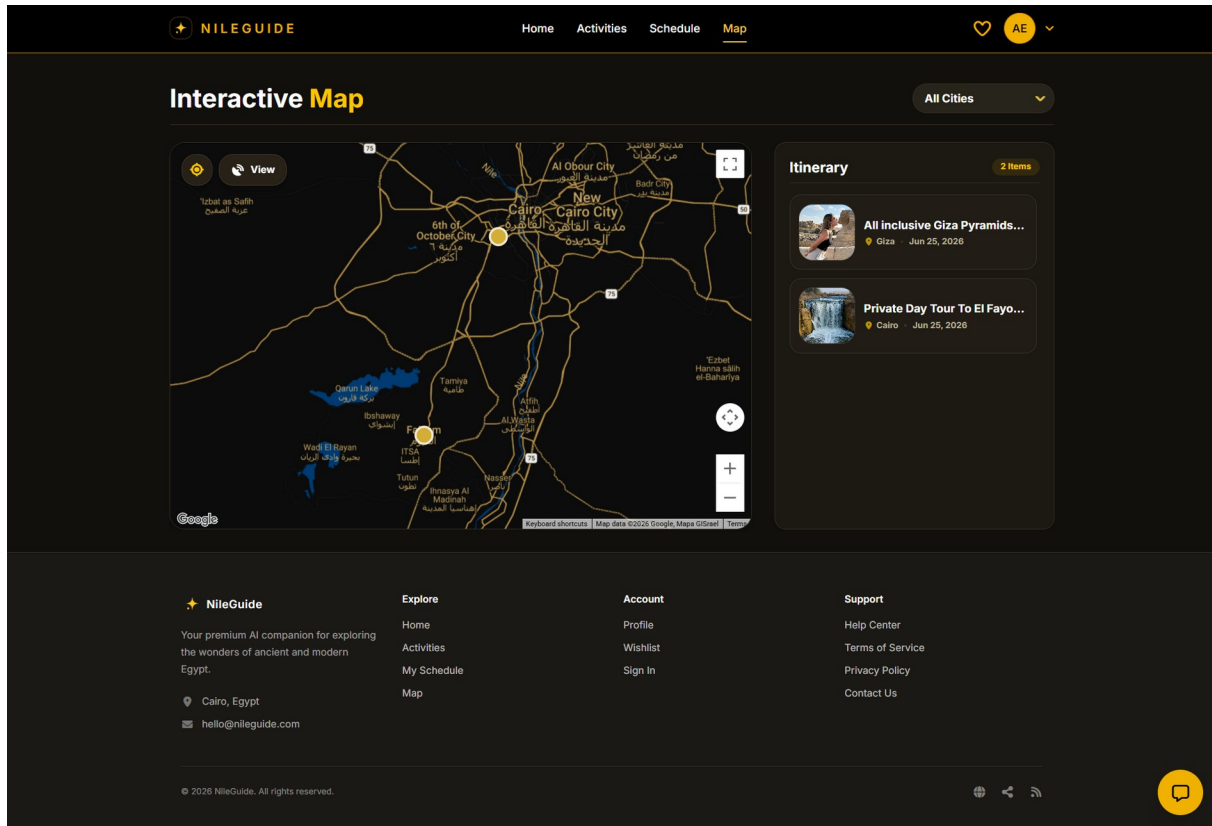


Figure 17. Map page

6.12 Profile page

The profile page collects the information used to personalize the tourist experience. The screen shows the profile avatar with initials and a camera button, name, email, date of birth, nationality/where the user is visiting from, travel-date option, preferred cities and interest categories. The Save My Profile button submits these choices. The final implementation focuses on cities, categories and travel information; budget is not shown in this final profile screen.

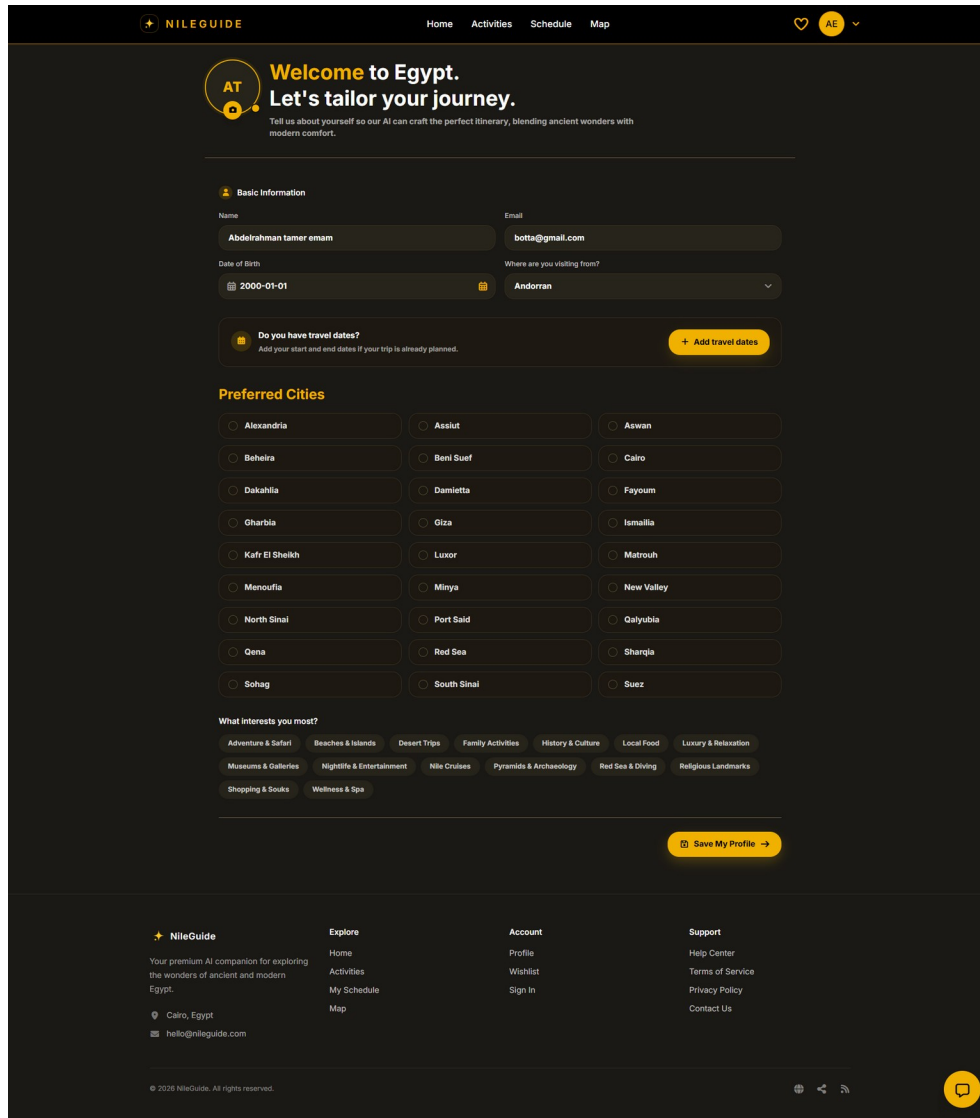


Figure 18. Profile page

6.13 User chatbot panel

This floating NileGuide AI panel is available from the user side. The screenshot shows the chatbot header, new-chat/history/close controls, a user message, the assistant welcome message and the input box for asking about Egypt. From the system side, the frontend sends the question to the FastAPI RAG service, which answers from the uploaded knowledge base and project data when relevant content exists.

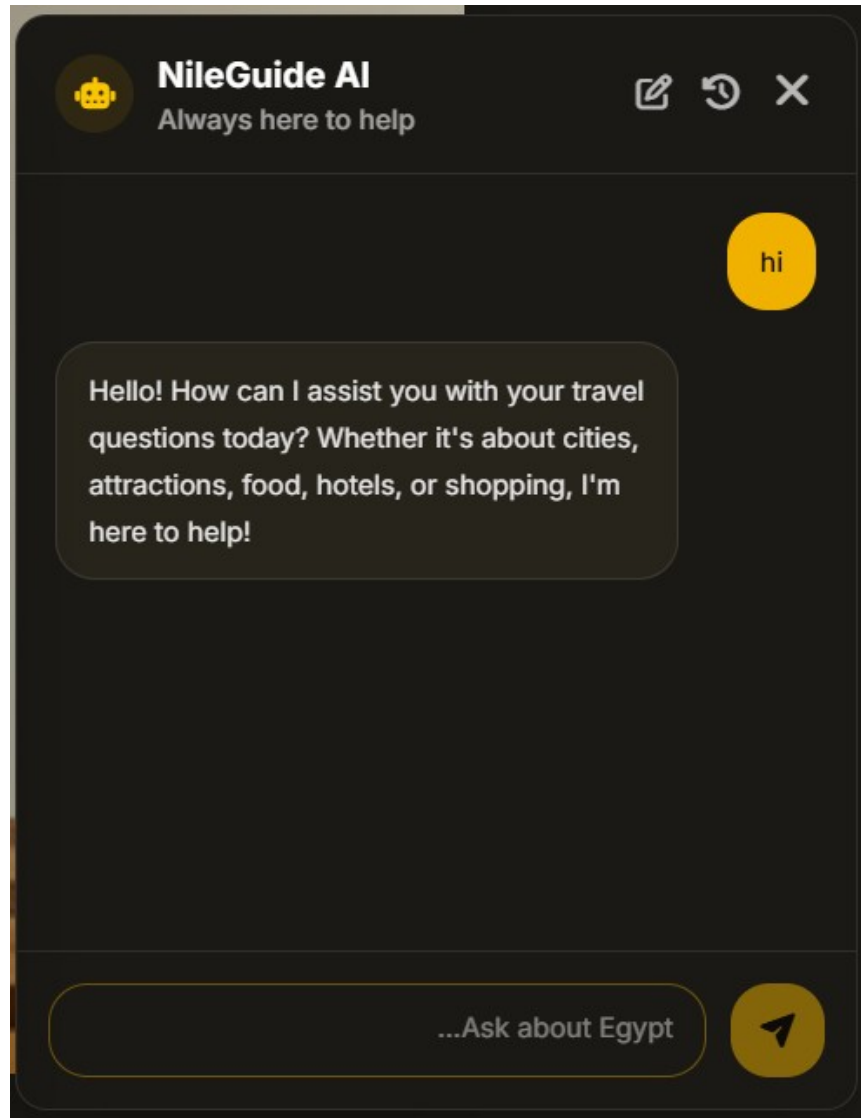


Figure 19. User chatbot panel

6.14 Chat history panel

The chat history drawer shows previous conversations saved for the user. In the screenshot, the user can start a New chat, reopen an older chat titled by the first message, or delete that conversation. This screen documents that the chatbot is not only a single message box; it has a conversation-management UI connected to the implemented chat conversation flow.

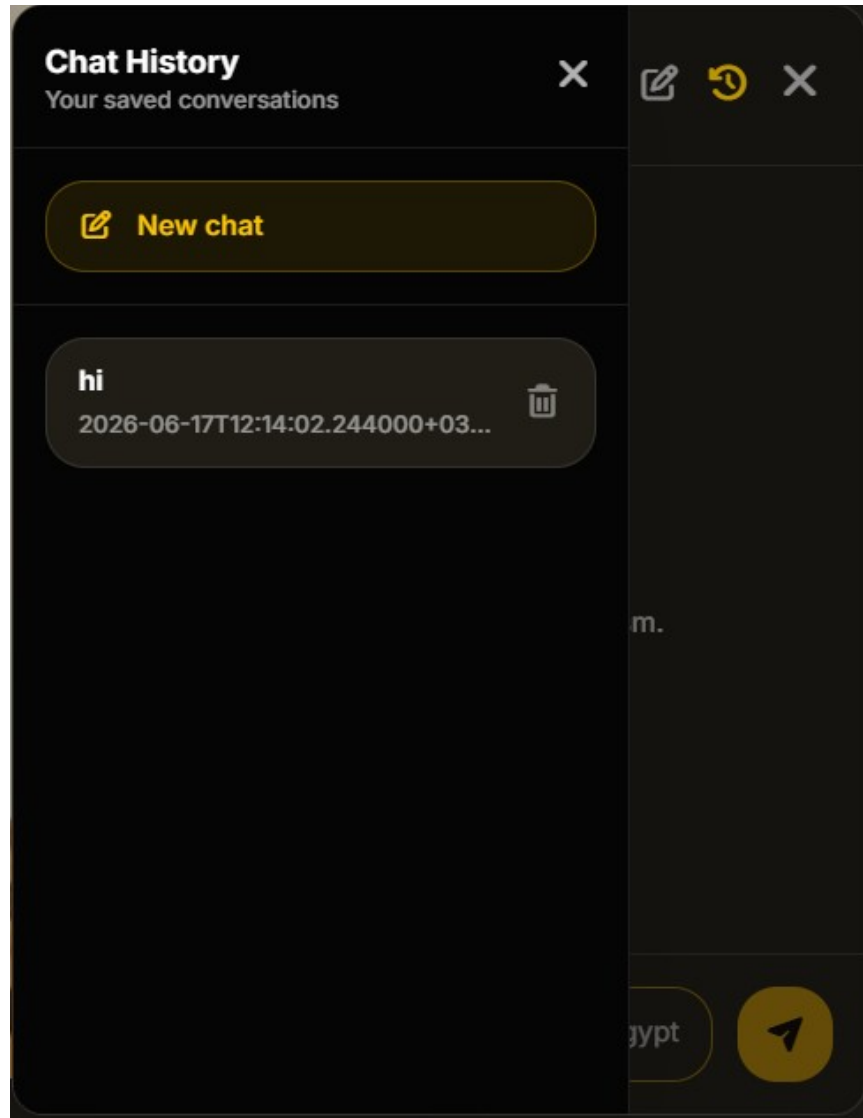


Figure 20. Chat history panel

6.15 Admin dashboard and users management

This screenshot shows the admin dashboard layout while the Manage Users module is selected. The top cards show current platform numbers such as total users, total activities, reviews, wishlist items and average rating. The users section includes search, role filter, status filter, Add New User and a users table showing name, email, role, join date, wishlist count, status and action icons. This page is protected for admin users only.

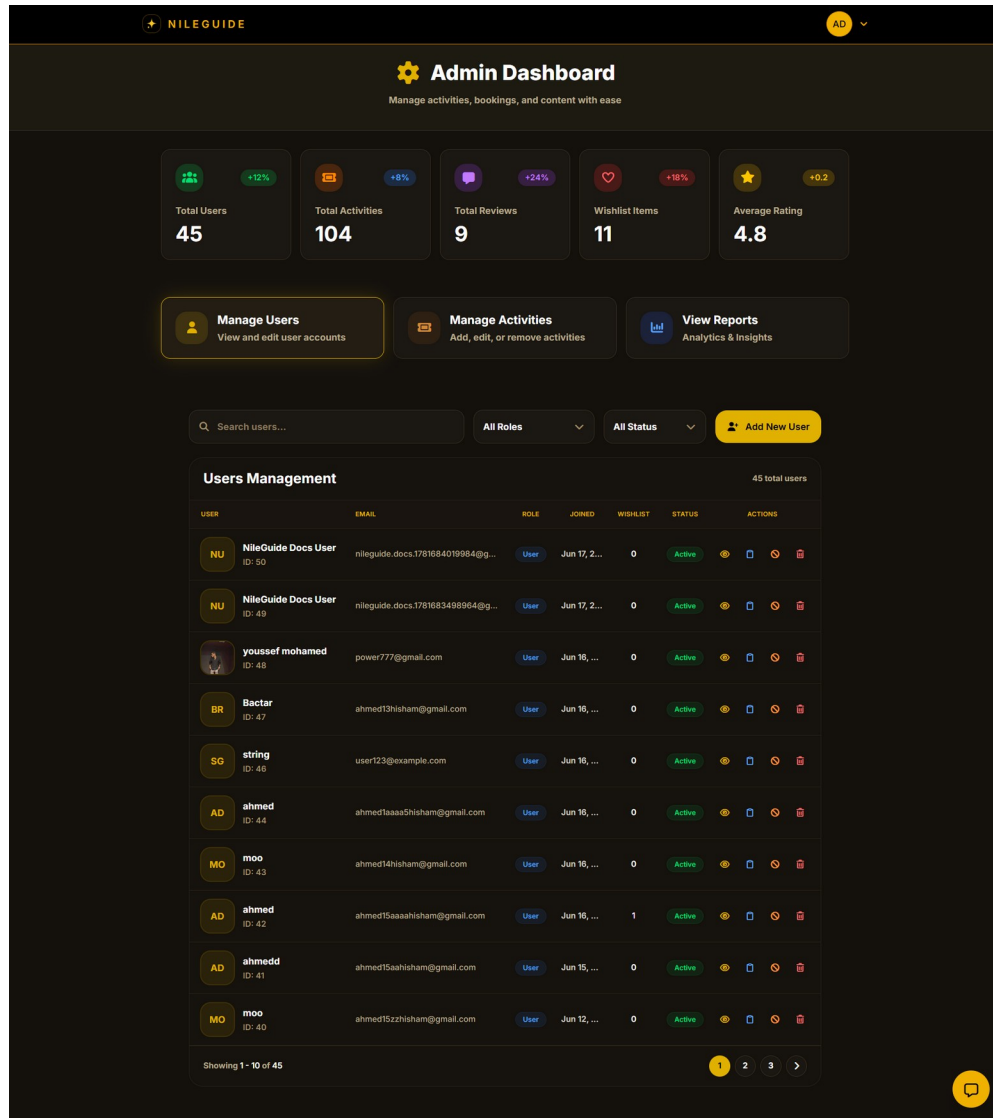


Figure 21. Admin dashboard and users management

6.16 Admin activities management

This screen shows the admin activity catalogue management module. The admin can search activities, filter by category, city and status, and click Add New Activity. The table displays activity image, title, description preview, category, price, location, active status and actions. The action icons represent viewing details, editing and deleting. This is the operational screen used to maintain the activities that tourists see on the public activities page.

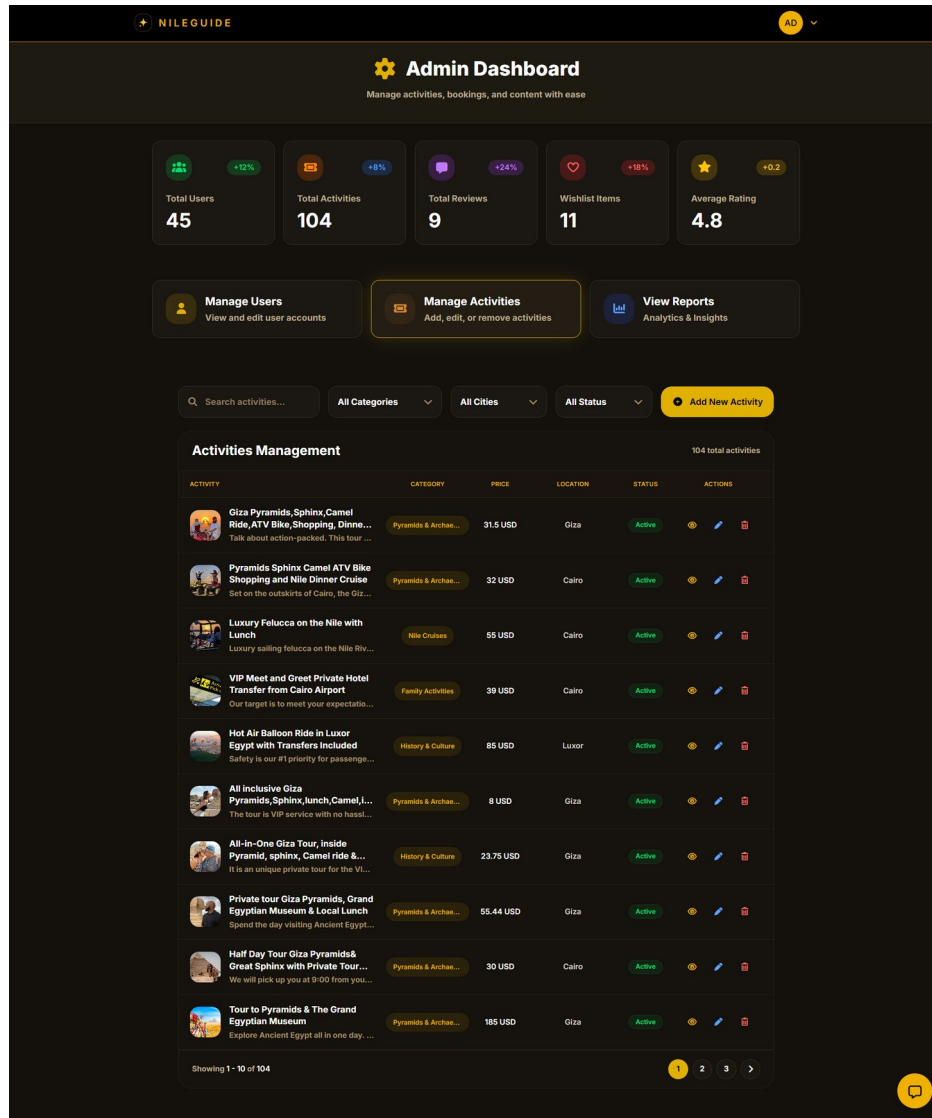


Figure 22. Admin activities management

6.17 Admin reports page

The reports screen keeps the same admin dashboard summary at the top and opens the View Reports module. The visible report area includes a Print Report button and chart sections such as Activity Views for the last 7 days and User Growth below it. These charts are used to give the admin a quick understanding of platform activity. When the database has limited tracking data, some charts may appear low or empty, which is expected during testing.

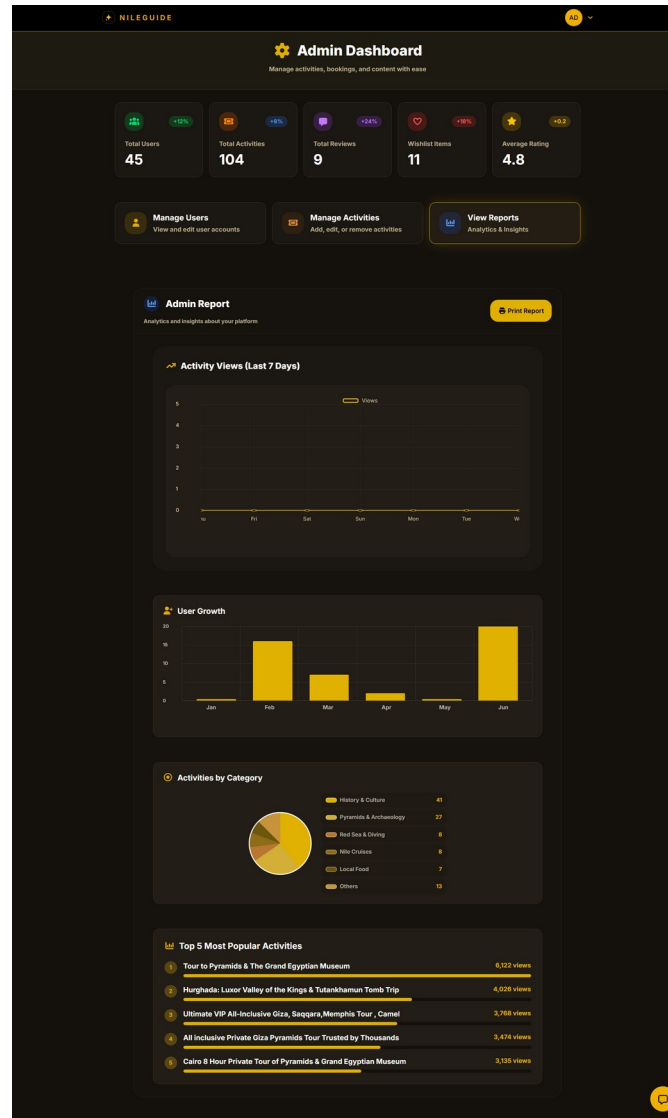


Figure 23. Admin reports page

6.18 Admin AI files upload panel

This panel is the admin entry point for maintaining the AI knowledge base. The visible state focuses on uploading files: the admin can drag files into the drop zone or browse from the device, then confirm the selected files. The panel accepts different document formats so the RAG service can extract text, split it into chunks, create embeddings and make the content available for chatbot answers.

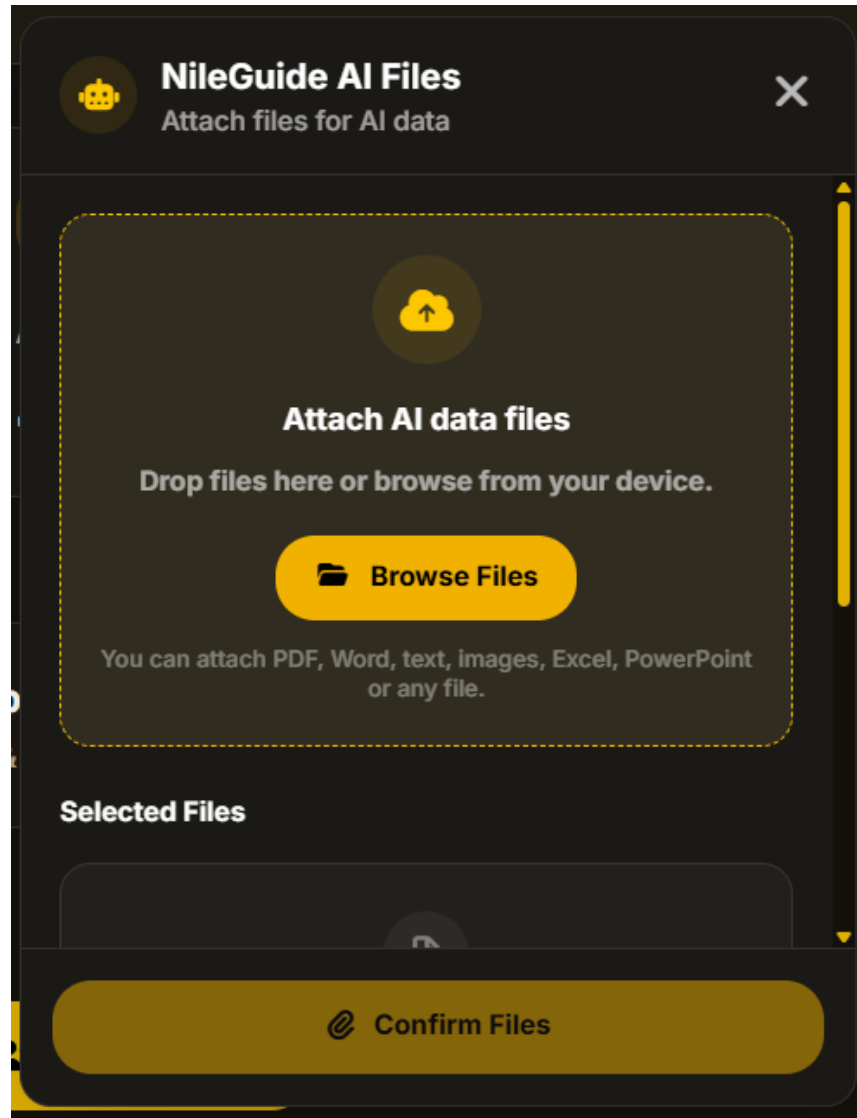


Figure 24. Admin AI files upload panel

6.19 Contact page

The contact page is a public support screen. The user can send a message by entering full name, email, subject and message. The right side shows support information, including expected response time and the official support email nileguide.noreply@gmail.com. This page completes the public-facing experience by giving users a direct communication channel with the platform team.

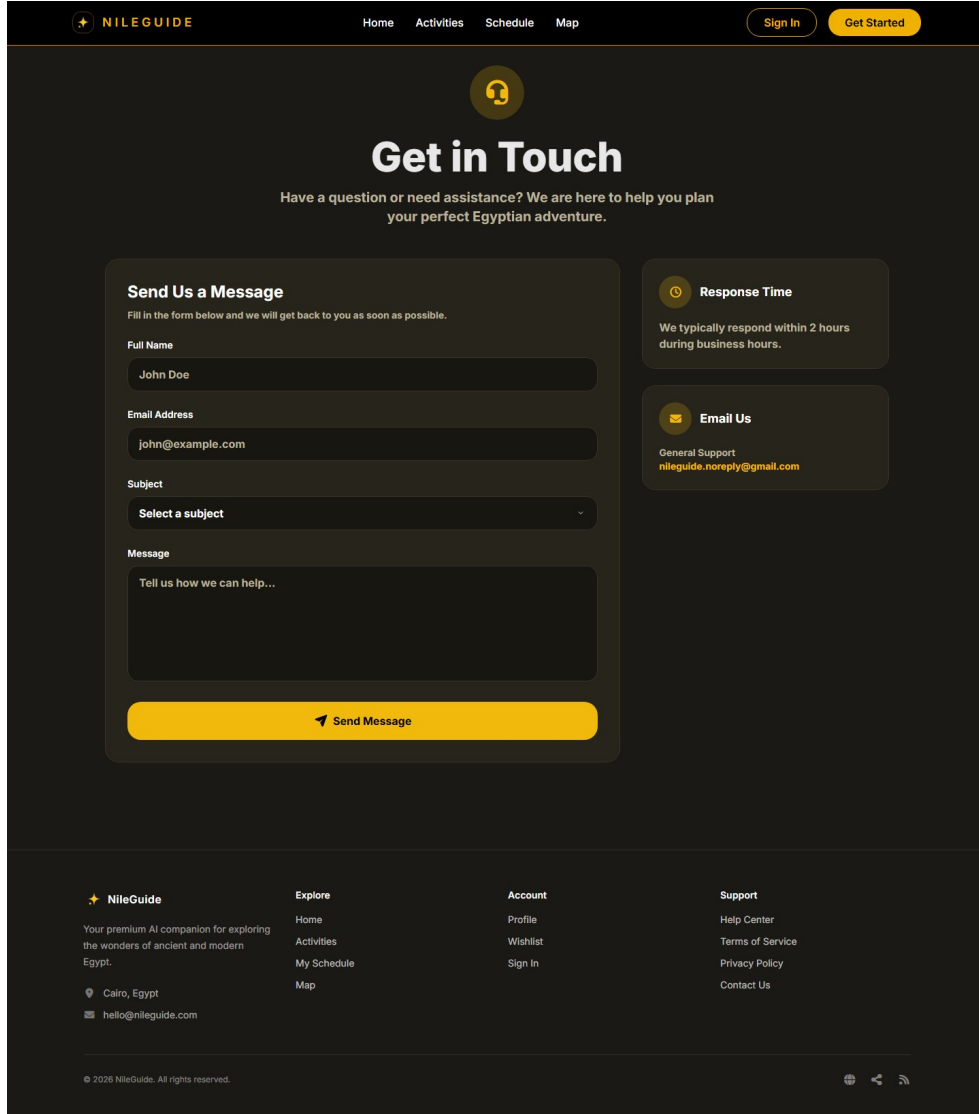


Figure 25. Contact page

6.20 Discussion of the Final Outcome

The final outcome satisfies the main project direction: a tourist can discover, save and plan activities, while the administrator can maintain content and AI knowledge. The largest change from the first-term plan is the AI part. Instead of trying to implement a broad recommender immediately, the team delivered a narrower and more controllable RAG assistant. This is more useful for the current system because the admin can update the knowledge base without redeploying the application.

Another important outcome is the separation between user features and admin features. The tourist pages focus on planning and discovery; the admin pages focus on operations. This separation made the routing, guards and documentation clearer during implementation.

Chapter 7

Conclusion & Future Work

7.1 Conclusion

NileGuide was completed as a web-based tourism planning platform with a real frontend, backend, database and AI/RAG module. The project moved beyond the first-term proposal by implementing concrete flows: tourist authentication, activities browsing, dynamic details, wishlist, schedule, map, profile, admin dashboard, activity management, user management, reports and AI file management. The system uses Angular for the UI, ASP.NET Core for the main API, SQL Server for relational data, Cloudinary for image storage, and FastAPI with MongoDB and Qdrant for RAG operations.

The final project also shows that a graduation system can be documented honestly: implemented features are described as implemented, changed features are explained, and future ideas are separated from the delivered release. This makes the system easier to present and defend during discussion.

7.2 Future Work

- Add a full personalized recommendation engine that combines profile preferences, budget, cities, ratings and schedule constraints.
- Add real booking integration or payment only after selecting a trusted provider API and defining a clear business flow.
- Add a full schedule conflict detector and optimizer that prevents overlapping activities based on duration and travel time.
- Add more report metrics and ensure activity-view tracking is complete for every details-page visit.
- Add refresh-token and Remember Me as a fully implemented secure session flow if long sessions are required.
- Improve the AI assistant with better source display, admin tagging of documents by city/category, and feedback collection.
- Add multilingual support, especially Arabic, for tourists and local users.

Chapter 8

References

- [1] Chang, H.-T., Chang, Y.-M., & Tsai, M.-T. (2016). ATIPS: Automatic Travel Itinerary Planning System for Domestic Areas. *Computational Intelligence and Neuroscience*.
- [2] Hassannia, R., Vatankhah Barenji, A., Li, Z., & Alipour, H. (2019). Web-Based Recommendation System for Smart Tourism: Multiagent Technology. *Sustainability*, 11(2), 323.
- [3] Srisawatsakul, C., & Boontarig, W. (2020). Tourism Recommender System using Machine Learning Based on User Public Instagram Photos. *IEEE InCIT 2020*.
- [4] Kumar, K., Birla, V., Sasi, A., & Deep, A. (2020). Design and Development of Travel and Tourism Recommendation System using Web-Scraped Data Positioned on Artificial Intelligence and Machine Learning. *International Journal of Advanced Trends in Computer Science and Engineering*, 9(4), 5670-5679.
- [5] Microsoft. ASP.NET Core Documentation. <https://learn.microsoft.com/aspnet/core/>
- [6] Angular Documentation. <https://angular.dev/>
- [7] FastAPI Documentation. <https://fastapi.tiangolo.com/>
- [8] Qdrant Documentation. <https://qdrant.tech/documentation/>
- [9] MongoDB Documentation. <https://www.mongodb.com/docs/>
- [10] Entity Framework Core Documentation. <https://learn.microsoft.com/ef/core/>

Chapter 9

Appendices

Appendix A - Final Feature Checklist

Area	Items	Status
Public pages	Home, login, register, forgot password, reset password, contact	Implemented
Tourist pages	Activities, details, wishlist, schedule, map, profile, chatbot	Implemented
Admin pages	Users, activities, reports, AI file panel	Implemented
AI/RAG	File upload/list/update/delete and chat response with sources	Implemented
Payment/booking confirmation	Not included in the current release	Future work
Remember Me / refresh-token UX	Not documented as delivered	Future work

Table 12. Final checklist

Appendix B - Main Project Folders

Project part	Important folders/files
Frontend	src/app/features, src/app/core, app.routes.ts, feature services and components
Backend	Controllers, DTOs, Models, Data, Services, Middleware, Options, Hubs
AI/RAG	routes, controllers, models, stores, helpers and vector database provider code

Table 13. Main code organization

Appendix C - Generated Figures and Tables Index

Figures

- Figure 1. NileGuide high-level system architecture
- Figure 2. Tourist workflow from discovery to chatbot support
- Figure 3. Admin workflow through dashboard modules
- Figure 4. JWT authentication and protected request sequence
- Figure 5. Simplified ERD for the main SQL Server entities
- Figure 6. RAG implementation flow for upload and chat

Figure 7. Deployment structure for NileGuide components

Figure 8. Home page

Figure 9. Login page

Figure 10. Register page

Figure 11. Forgot password page

Figure 12. Reset password page

Figure 13. Activities page

Figure 14. Activity details page

Figure 15. Wishlist page

Figure 16. Schedule page

Figure 17. Map page

Figure 18. Profile page

Figure 19. User chatbot panel

Figure 20. Chat history panel

Figure 21. Admin dashboard and users management

Figure 22. Admin activities management

Figure 23. Admin reports page

Figure 24. Admin AI files upload panel

Figure 25. Contact page

Tables

Table 1. Abbreviations used in the NileGuide report

Table 2. Final implementation scope compared with the first-term plan

Table 3. Summary of related tourism planning and recommendation work

Table 4. System roles and responsibilities

Table 5. Functional requirements implemented or represented in the final version

Table 6. Frontend implementation summary

Table 7. Angular routes used by the final frontend

Table 8. Backend implementation summary

Table 9. Main ASP.NET Core API endpoints

Table 10. FastAPI RAG endpoints

Table 11. Representative manual test cases

Table 12. Final checklist

Table 13. Main code organization